4.1 Gaps and Omissions in the Testing Process

This section evaluates the limitations of the testing approach documented in LO3, assessing their significance for a safety-critical medical drone delivery system and prioritising them by potential impact.

Critical Gaps (High Impact)

Absence of mutation testing: While LO3 reports 92%-line coverage and 81% branch coverage, these metrics measure code execution, not test effectiveness. High coverage can mask weak assertions that execute code paths without verifying correctness. For MedSupplyDrones, this is a critical gap: a test that executes the no-fly zone avoidance logic but uses assertions too loose to detect boundary errors provides false confidence. Research suggests mutation scores of 70-80% are typical for well-tested codebases; without PIT analysis, we cannot verify our tests would detect introduced faults. **Severity: Critical** - undetected faults in pathfinding could result in drones entering restricted airspace.

Designer bias in test creation: As the sole developer who designed, implemented, and tested the system, I inherently tested scenarios I anticipated rather than discovering truly unexpected edge cases. This single-perspective testing violates the principle that testers should be independent from developers. The 29 JSON test fixtures in src/test/resources/ reflect my mental model of failure modes, which may systematically exclude scenarios I did not conceive. **Severity: Critical** systematic blind spots cannot be quantified without independent review or automated exploratory testing.

Significant Gaps (Medium Impact)

Branch coverage deficit: The 81% branch coverage reported in LO3 means approximately 67 of 355 conditional branches remain unexercised. Analysis of the coverage report reveals these are concentrated in exception handling paths within DroneRoutingService (network timeouts, malformed API responses) and defensive null-checks in GeometryService. For a system handling emergency medical deliveries, error-handling code is not optional, it executes precisely when reliability matters most. **Severity: Medium-High** untested error paths could cause silent failures during API outages.

External API coupling in integration tests: FullStackIntegrationTest depends on the live ILP REST API, creating non-deterministic test behaviour. This violates test isolation principles and makes the test suite unsuitable for CI/CD pipelines where external service availability cannot be guaranteed. More critically, if the ILP API changes its response schema, tests may fail or pass incorrectly, masking real defects. **Severity: Medium** affects test reliability and deployment confidence rather than production safety.

Implicit algorithm testing: The A* pathfinding algorithm is tested only indirectly through DroneRoutingService output validation. If the routing service were refactored to use a different algorithm (Dijkstra, D*), existing tests would not verify the new implementation correctly. This coupling between algorithm and service violates modularity principles and makes the test suite fragile to architectural changes. **Severity: Medium** affects maintainability and refactoring confidence.

Minor Gaps (Lower Impact)

Load testing absence: While individual requests complete under 400ms (well within the 30-second budget), there is no systematic testing of concurrent request handling. During emergency surge conditions (e.g., natural disaster requiring multiple simultaneous dispatches), thread contention or resource exhaustion could degrade performance. **Severity: Low-Medium** relevant for production deployment but not for coursework demonstration.

Hardcoded coordinate fixtures: Test fixtures use manually selected coordinates within Edinburgh's operational boundary. This risks missing edge cases at unusual coordinate combinations or boundary conditions that manual test design did not anticipate. **Severity: Low** mitigated by boundary value analysis already performed in GeometryCalculationsTest.

4.2 Target Coverage and Performance Levels

This section establishes and justifies target levels for both structural coverage metrics and measurable quality attributes, distinguishing between aspirational targets and realistic expectations given coursework constraints.

Structural Coverage Targets

Class coverage - Target: 100%, Realistic: 95%+. Full class coverage ensures no dead code exists and every component is exercised at least once. The realistic target acknowledges that Spring Boot bootstrap classes (Application.main()) are difficult to instrument meaningfully in unit tests.

Method coverage - Target: 100%, Realistic: 95%+. Complete method coverage verifies all public interfaces are tested. The gap accounts for defensive getter/setter methods and debugging utilities that provide limited value when tested in isolation.

Line coverage - Target: >90%, Realistic: 90%+. The 90% threshold balances thoroughness against diminishing returns from testing exception-only code paths. Industry benchmarks for safety-critical systems typically require 80-90% line coverage (DO-178C Level C equivalent).

Branch coverage - Target: 85%, Realistic: 80%+. Branch coverage is more demanding than line coverage as it requires exercising both outcomes of every conditional. The 85% target reflects the need to test error-handling branches, while acknowledging that some defensive code (null checks on validated input) may be impractical to exercise without fault injection.

Measurable Quality Attributes

Performance - Target: <30 seconds, Aspirational: <1 second. The 30-second budget is mandated by the ILP coursework specification. However, for a real-time emergency dispatch system, sub-second response times would be necessary to support rapid decision-making. The aspirational target drives optimisation beyond minimum compliance.

Determinism - Target: 100% reproducibility. Identical inputs must produce byte-identical outputs across repeated executions. Non-determinism would make debugging impossible and violate the principle that route planning should be

predictable. This is verified by executing identical requests multiple times and comparing JSON structures.

Geometric precision - Target: 10^12 tolerance maintained throughout calculations. The EPS constant (10^12) must be sufficient to prevent floating-point comparison errors while avoiding false positives from accumulated rounding. This is more challenging to verify than functional requirements because precision degradation may only manifest over long path sequences.

Mutation score - Target: 80%+, Currently: Unknown. A mutation score of 80%+ would indicate that tests detect 4 out of 5 artificially introduced faults. Without PIT analysis, this critical quality attribute remains unmeasured, representing the most significant gap in test suite validation.

4.3 Comparison of Testing Results with Targets

This section compares achieved results (documented in LO3) against the targets established above, explaining variations and assessing whether shortfalls represent acceptable trade-offs given resource constraints.

Coverage Metrics Assessment

Class coverage: 95% achieved vs 100% target. (Acceptable) The 2 uncovered classes are the Application bootstrap class (testing provides no value) and a data utility class exercised only through integration paths. Achieving 100% would require testing Spring Boot lifecycle methods, which validates framework behaviour rather than application logic.

Method coverage: 95% achieved vs 100% target (Acceptable). Uncovered methods (7 of 141) are debugging utilities, toString() implementations, and defensive null-guards. Testing these in isolation would increase coverage without improving defect detection.

Line coverage: 92% achieved vs >90% target. Target met. The 54 uncovered lines (8%) are concentrated in exception handling blocks that require fault injection to exercise. This represents acceptable residual risk for coursework scope.

Branch coverage: 81% achieved vs 85% target Below target - requires attention. The 67 uncovered branches are primarily in error-handling code within DroneRoutingService and IlpRestService. For a safety-critical system, this gap is concerning, error paths execute during failures when correct behaviour is most important. This shortfall should be prioritised for remediation.

Quality Attributes Assessment

Performance: <400ms achieved vs <30s target. Target exceeded by 98.7%. All pathfinding operations complete in under 400ms, providing substantial margin against the specification requirement. This exceeds even the aspirational <1s target for real-time dispatch.

Determinism: 100% achieved vs 100% target. Target met. Repeated execution of identical requests produces byte-identical JSON outputs, verified across 5 repetitions per test case in NonFunctionalTest.

Geometric precision: 10^12 tolerance-maintained vs 10^12 target. Target met for individual calculations. However, cumulative precision over paths exceeding 100

moves has not been systematically verified. This represents residual uncertainty rather than confirmed failure.

Mutation score: Unknown vs 80% target. Cannot assess - critical gap. Without mutation testing, we cannot verify that high coverage translates to effective fault detection. This is the most significant unmeasured quality attribute.

Summary: Are Targets Realistic?

The targets established in section 4.2 were largely realistic given coursework constraints. The 5% shortfalls in class and method coverage represent pragmatic trade-offs, not failures. However, the 85% branch coverage target may have been optimistic without dedicated fault injection infrastructure. The performance target was conservative — actual results demonstrate significant optimisation headroom exists. The absence of mutation score measurement means one critical target remains unverifiable, which is a process gap rather than a coverage failure.

4.4 Achieving Target Levels: Remediation Plan

This section proposes specific improvements to address the gaps identified in sections 4.1 and 4.3, prioritised by impact and estimated effort. Each improvement is linked to the specific deficiency it addresses.

Priority 1: Mutation Testing Integration

Addresses: Absence of mutation testing (4.1), unknown mutation score (4.3)

Implementation: Integrate PIT (pitest.org) Maven plugin with configuration targeting the uk.ac.ed.acp.cw2 package. Configure mutators for arithmetic, conditional boundary, and return value mutations. Set target mutation score threshold of 80%.

Effort estimate: 2-3 hours for PIT configuration and initial run; 4-8 hours to strengthen tests based on surviving mutants. This is the highest-value improvement as it validates existing test effectiveness.

Priority 2: Branch Coverage Improvement

Addresses: Branch coverage deficit (4.1), 81% vs 85% target (4.3)

Implementation: Use IntelliJ coverage reports to identify specific uncovered branches. Create targeted tests using Mockito.when().thenThrow() to exercise exception handlers in DroneRoutingService and IlpRestService. Add tests for defensive null-checks using null input parameters.

Effort estimate: 3-4 hours to analyse coverage gaps and implement around 15-20 additional test cases targeting specific branches.

Priority 3: API Stubbing for Test Isolation

Addresses: External API coupling (4.1)

Implementation: Implement WireMock stubs for ILP REST API endpoints (/drones, /service-points, /restricted-areas). Record actual API responses as baseline fixtures. Configure FullStackIntegrationTest to use WireMock server. Maintain separate LiveApiValidationTest for periodic verification against real API.

Effort estimate: 4-6 hours for WireMock setup and response recording. Provides deterministic CI/CD behaviour and faster test execution.

Priority 4: Property-Based Testing

Addresses: Designer bias (4.1), hardcoded coordinate fixtures (4.1)

Implementation: Implement jqwik property tests for GeometryService with randomised coordinate generation within Edinburgh's operational boundary (approximately 55.9°-56.0°N, 3.1°-3.3°W). Define invariants: distance is always non-negative, closeness is symmetric, point-in-polygon is consistent with winding number algorithm.

Effort estimate: 3-5 hours to implement property generators and invariant definitions. Discovers edge cases that manual test design would miss.

Priority 5: Direct Algorithm Testing

Addresses: Implicit algorithm testing (4.1)

Implementation: Extract A* pathfinding into a separate PathfindingAlgorithm interface. Create AStarPathfindingTest with unit tests for: optimal path on simple graphs, handling of blocked nodes, path cost calculation, and empty graph edge cases. This enables future algorithm substitution without test suite changes.

Effort estimate: 2-3 hours for interface extraction and unit test creation. Improves modularity and maintainability.

Effort Summary

Total estimated effort for all improvements: 14-26 hours. For coursework constraints, priorities 1 and 2 (mutation testing + branch coverage) provide the highest value within approximately 10 hours of work. Priorities 3-5 would be essential for production deployment but represent reasonable scope limitations for academic assessment.

Conclusion

The testing evaluation reveals a suite that achieves strong structural coverage (95% class, 95% method, 92% line) and excellent performance (<400ms against 30s budget), but with significant gaps in test effectiveness validation. The most critical limitation is the absence of mutation testing, which means high coverage metrics may not translate to high fault-detection capability. The 81% branch coverage, while respectable, falls short of the 85% target and leaves error-handling paths - critical for reliability, inadequately tested.

For a safety-critical medical drone delivery system, these gaps would require remediation before production deployment. The prioritised improvement plan addresses deficiencies in order of impact, with mutation testing and branch coverage improvement providing the highest value within reasonable effort. The designer bias limitation is structural and cannot be fully addressed without independent testing resources, but property-based testing offers partial mitigation by automating exploratory test generation.

The evaluation demonstrates that coverage metrics alone are insufficient to assess test suite quality. A mature testing process requires both high coverage and validation that tests effectively detect faults, a distinction that motivates the emphasis on mutation testing as the highest-priority improvement.